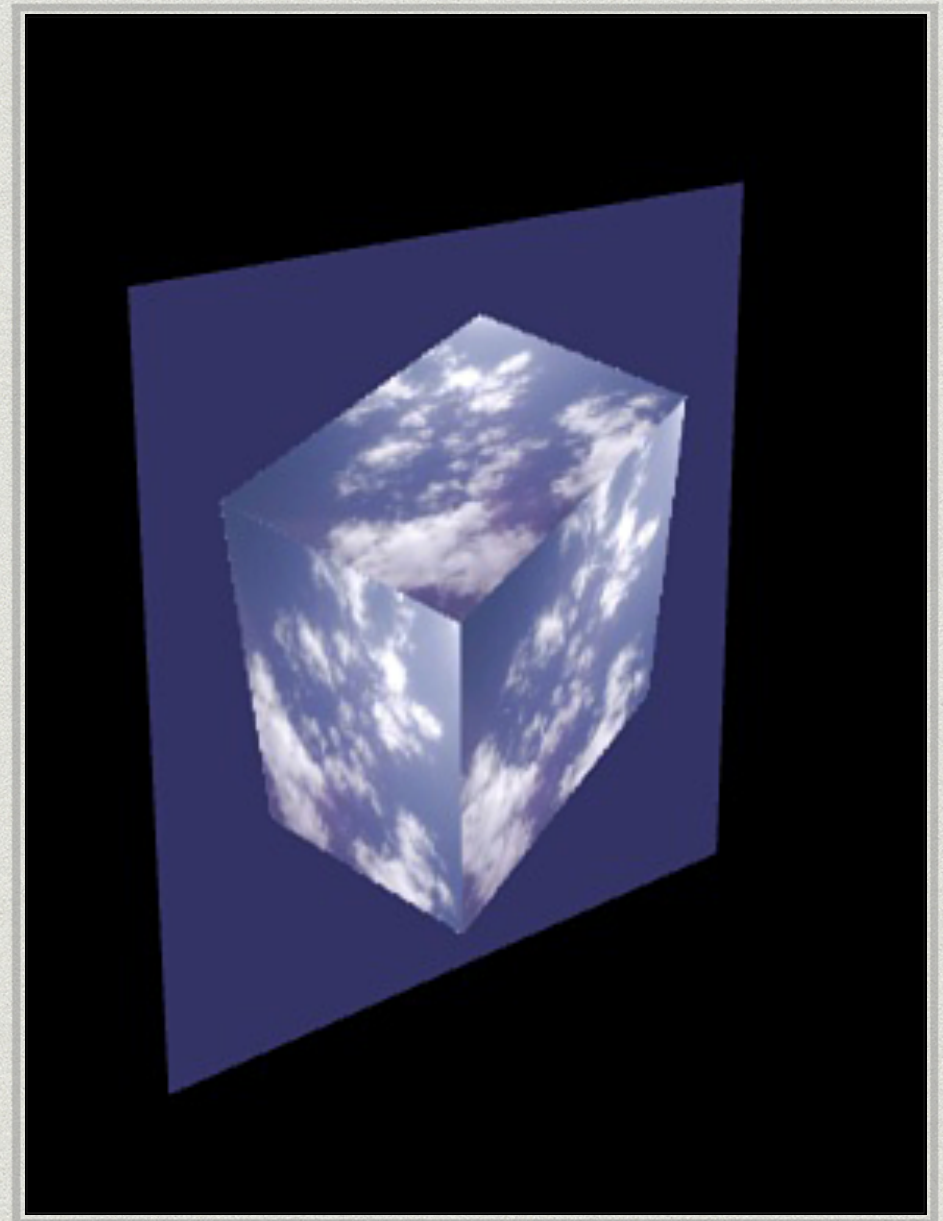


FBO: THE FRAME BUFFER OBJECT

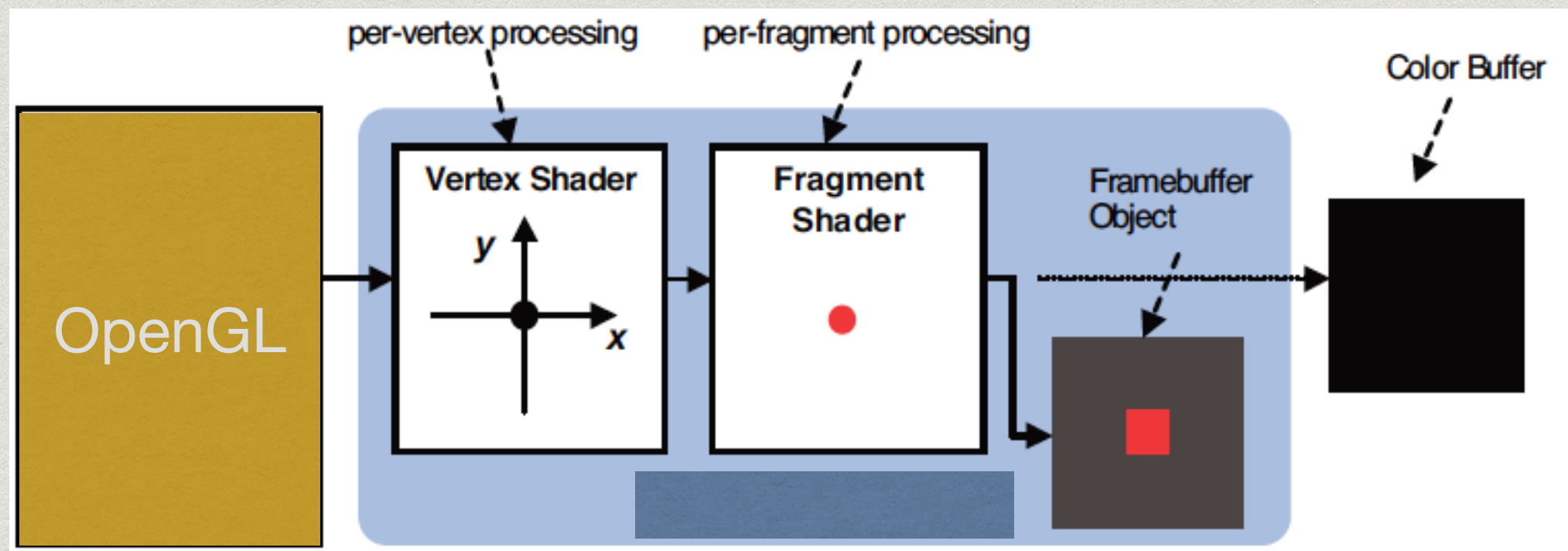
Offscreen Drawing: Drawing to memory

- * Use the rendered image as texture
- * Why is it needed?
 - * Generating images
 - * Debugging
 - * Applying image effects using GPU
 - * Shadowing (will be discussed in the next class)



Offscreen Drawing: Drawing to memory

- * Where to? To texture image
- * Framebuffer Object

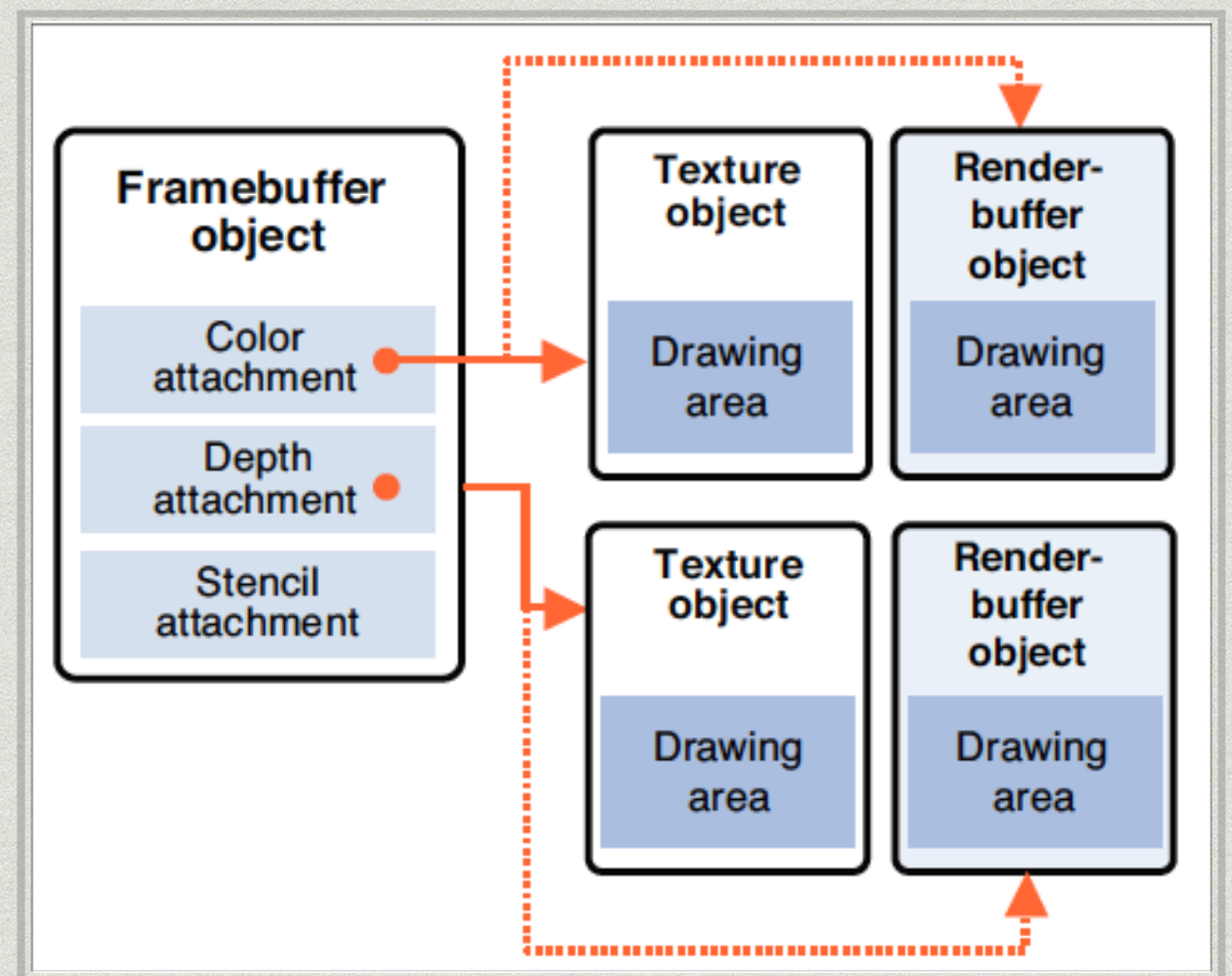


Framebuffer Object (FBO)

- * Buffers in OpenGL
 - * Color buffer
 - * Depth buffer
 - * Stencil buffer
- * Framebuffer Object

Framebuffer Object

- * Substitutes buffers in OpenGL
- * Buffers in OpenGL
 - * Color buffer
 - * Depth buffer
 - * Stencil buffer

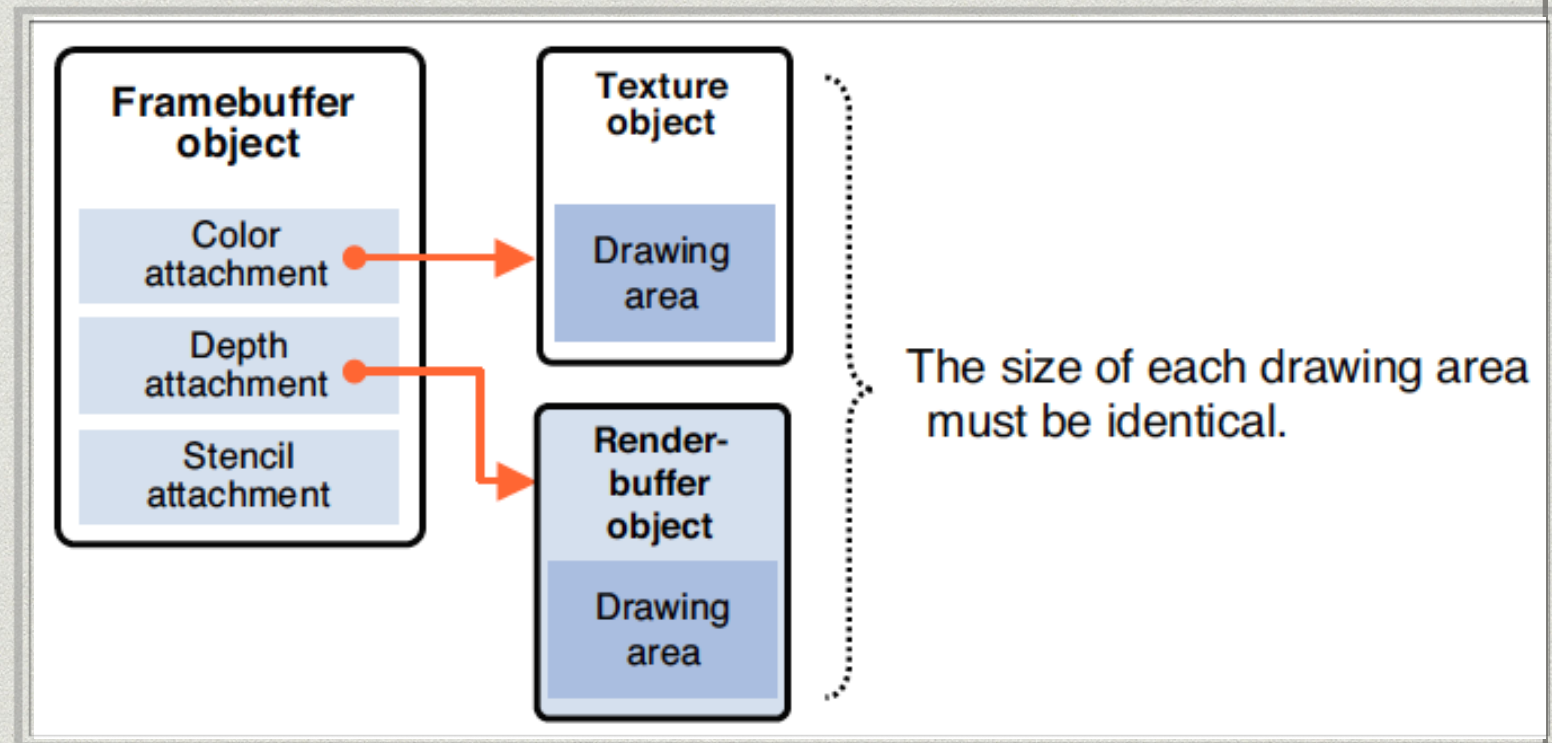


Framebuffer Object

- * Target type
 - * Texture Object
 - * Texture image in video memory
- * Renderbuffer Object
 - * General memory target

Drawing into texture image

- * Attach color and depth buffer
- * Color buffer as
- * Depth buffer as



Steps

- * 1. Create a framebuffer object
- * 2. Create a texture object and set its size and parameters
 - * Create/Bind/TexImage2D/Parameters
- * 3. Create a renderbuffer object
 - * Create/Bind/Storage
- * 4. Attach
 - * Bind/Texture2D/Renderbuffer
- * 5. Check
- * 6. Draw using the framebuffer object

Step 1. Create Framebuffer Object

- * GLunit fbo;
- * glGenFramebuffers(1, &fbo);
- * glBindFramebuffer(GL_FRAMEBUFFER, fbo);
- * Note: glBindFramebuffer(GL_FRAMEBUFFER,0)

Name

glGenFramebuffers — generate framebuffer object names

C Specification

```
void glGenFramebuffers( GLsizei n,  
                        GLuint *ids );
```

Parameters

n

Specifies the number of framebuffer object names to generate.

ids

Specifies an array in which the generated framebuffer object names are stored.

Name

glDeleteFramebuffers — delete framebuffer objects

C Specification

```
void glDeleteFramebuffers( GLsizei n,  
                           GLuint *framebuffers );
```

Parameters

n

Specifies the number of framebuffer objects to be deleted.

framebuffers

A pointer to an array containing *n* framebuffer objects to be deleted.

Step 2. Create Texture object

- * GLuint tex;
- * glGenTextures(1, &tex);
- * glBindTexture(GL_TEXTURE_2D, tex);
- * glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, OFFSCREEN_WIDTH, OFFSCREEN_HEIGHT, 0, GL_RGBA, GL_UNSIGNED_BYTE, null);
- * glTexParameteri(...s);

Step 3. Create Renderbuffer Object

- * `GLuint rbo;`
- * `glGenRenderbuffers(1, &rbo);`
- * `glBindRenderbuffer(GL_RENDERBUFFER, rbo);`
- * `glRenderbufferStorage(GL_RENDERBUFFER,
GL_DEPTH24_STENCIL8, OFFSCREEN_WIDTH,
OFFSCREEN_HEIGHT);`

Step 4. Attach

- * Set texture object to Framebuffer object
- * `glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, tex, 0);`
- * `glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_STENCIL_ATTACHMENT, GL_RENDERBUFFER, rbo);`

Step 5. Check

GLenum **glCheckFramebufferStatus**(GLenum *target*);

- GL_FRAMEBUFFER_UNDEFINED is returned if the specified framebuffer is the default read or draw framebuffer, but the default framebuffer does not exist.
- GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT is returned if any of the framebuffer attachment points are framebuffer incomplete.
- GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT is returned if the framebuffer does not have at least one image attached to it.
- GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER is returned if the value of GL_FRAMEBUFFER_ATTACHMENT_OBJECT_TYPE is GL_NONE for any color attachment point(s) named by GL_DRAW_BUFFERi.
- GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER is returned if GL_READ_BUFFER is not GL_NONE and the value of GL_FRAMEBUFFER_ATTACHMENT_OBJECT_TYPE is GL_NONE for the color attachment point named by GL_READ_BUFFER.
- GL_FRAMEBUFFER_UNSUPPORTED is returned if the combination of internal formats of the attached images violates an implementation-dependent set of restrictions.
- GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE is returned if the value of GL_RENDERBUFFER_SAMPLES is not the same for all attached renderbuffers; if the value of GL_TEXTURE_SAMPLES is the not same for all attached textures; or, if the attached images are a mix of renderbuffers and textures, the value of GL_RENDERBUFFER_SAMPLES does not match the value of GL_TEXTURE_SAMPLES.
- GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE is also returned if the value of GL_TEXTURE_FIXED_SAMPLE_LOCATIONS is not the same for all attached textures; or, if the attached images are a mix of renderbuffers and textures, the value of GL_TEXTURE_FIXED_SAMPLE_LOCATIONS is not GL_TRUE for all attached textures.
- GL_FRAMEBUFFER_INCOMPLETE_LAYER_TARGETS is returned if any framebuffer attachment is layered, and any populated attachment is not layered, or if all populated color attachments are not from textures of the same target.

Step 6. Draw

- * 6-1: bind frame buffer
- * 6-2: set viewport/projection & view transformation matrix for texture image
- * 6-3: draw for fbo
- * 7-1: bind frame buffer as null
- * 7-x: draw normal scene

Step 6. Draw - 6-1/6-2

- * `glBindFramebuffer(GL_FRAMEBUFFER, fbo);`
- * `glClear(...); glViewport(...); .....`
- * `Draw(); // to be drawn to fbo`

Step 7. Draw

- * `glBindFramebuffer(GL_FRAMEBUFFER, 0);`
- * `glClear(...); glViewport(...); .....`
- * `Draw()` // to be drawn to screen

Exercise

- * Create a box
 - * The box has a screen: The screen shows the previous spline images
 - * Two shader programs
 - * 0 : for fbo drawing -> previous exercise
 - * 1: for screen drawing -> simple ones for this cases
- * (optional) Create depth buffer as texture image and draw it on another face

Summary

- * FBO
 - * Color buffer
 - * Depth buffer
 - * Stencil buffer
- * Target
 - * Texture
 - * Render buffer